

STREAMHUB

Plataforma Unificada de Analytics para Streamers

DOCUMENTO DE REQUISITOS — Versão 1.0

Classificação: Ideia Conceitual / Pré-MVP

Data: Abril de 2026

1. Resumo Executivo

StreamHub é uma plataforma web que centraliza os dados de analytics de múltiplas plataformas de streaming e redes sociais em um único dashboard inteligente. O objetivo é eliminar a necessidade de o criador de conteúdo acessar individualmente cada plataforma para acompanhar seu desempenho.

Hoje, um streamer que opera no YouTube, Twitch, Kick e TikTok precisa navegar entre quatro dashboards distintos, com linguagens visuais diferentes, métricas inconsistentes e sem nenhuma visão consolidada do seu negócio como criador. O StreamHub resolve esse problema conectando as APIs oficiais dessas plataformas via autenticação OAuth, agregando os dados e apresentando insights cruzados que nenhuma plataforma oferece individualmente.

O diferencial competitivo não está apenas na unificação — está na inteligência gerada a partir dos dados combinados: qual plataforma gera mais engajamento real, qual horário de live performa melhor no agregado, qual conteúdo converte espectadores em seguidores com maior eficiência.

2. Problema a Ser Resolvido

2.1 Contexto

O mercado de criadores de conteúdo ao vivo cresceu de forma acelerada entre 2020 e 2025. Streamers profissionais e semi-profissionais frequentemente operam em duas ou mais

plataformas simultaneamente para maximizar audiência e receita. Esse modelo multi-plataforma, porém, cria uma fragmentação operacional significativa.

2.2 Dores Identificadas

- **Tempo:** Perda de tempo navegando entre múltiplos dashboards diariamente.
- **Dados:** Ausência de visão consolidada de receita, seguidores e engajamento.
- **Análise:** Impossibilidade de comparar performance entre plataformas com métricas equivalentes.
- **Estratégia:** Decisões de conteúdo tomadas com base em dados parciais e isolados.
- **Mercado:** Nenhuma ferramenta atual resolve o problema de forma completa e acessível.

2.3 Público-Alvo

- Streamers com presença em 2 ou mais plataformas simultaneamente
- Criadores de conteúdo ao vivo em fase de crescimento (1K a 500K seguidores)
- Managers e agências que gerenciam múltiplos criadores

3. Escopo do Produto

3.1 Plataformas Suportadas — Fase 1 (MVP)

Plataforma	Viabilidade	Dados Disponíveis	Limitações
YouTube	✓ Alta	Views, inscritos, receita estimada, métricas de live	Receita exata não exposta
Twitch	✓ Alta	Viewers, follows, bits, sub count, VODs	Dados de receita parciais
Kick	● Média	Viewers, seguidores, clips	API jovem, sujeita a mudanças
TikTok	● Baixa	Seguidores, views de vídeo	Live analytics extremamente restritos

3.2 O Que Está Fora do Escopo (MVP)

- Automação de postagem ou agendamento de conteúdo
- Moderação de chat unificada
- Gestão de patrocinadores
- Receita financeira exata (valores precisos por plataforma)

4. Requisitos Funcionais

4.1 Autenticação e Conexão de Contas

- RF01 — O usuário deve poder conectar suas contas via OAuth oficial de cada plataforma suportada
- RF02 — O sistema deve armazenar tokens de acesso de forma segura com renovação automática
- RF03 — O usuário deve poder desconectar qualquer plataforma a qualquer momento
- RF04 — O sistema deve exibir o status de conexão de cada plataforma (ativo, expirado, erro)

4.2 Dashboard Principal

- RF05 — O dashboard deve exibir cards de resumo com as principais métricas de cada plataforma
- RF06 — O usuário deve poder personalizar quais métricas aparecem no dashboard
- RF07 — Os dados devem ser atualizados automaticamente em intervalos configuráveis
- RF08 — O dashboard deve exibir um score agregado de performance multi-plataforma

4.3 Analytics de Live

- RF09 — Durante uma live ativa, exibir viewers simultâneos de todas as plataformas em tempo real
- RF10 — Registrar histórico de pico de viewers, duração média e chat activity por live
- RF11 — Gerar relatório automático ao encerrar uma live com comparativo entre plataformas

4.4 Crescimento e Histórico

- RF12 — Exibir gráfico de evolução de seguidores/inscritos por plataforma e consolidado
- RF13 — Permitir comparação de períodos: 7 dias, 30 dias, 3 meses, 1 ano
- RF14 — Destacar dias e horários de maior crescimento orgânico

4.5 Insights Inteligentes

- RF15 — Identificar automaticamente qual plataforma tem melhor taxa de conversão de viewer para seguidor
- RF16 — Sugerir os melhores horários para live com base no histórico de performance
- RF17 — Alertar quando uma métrica cair abaixo de uma média histórica configurável
- RF18 — Gerar relatório semanal automático com os principais destaques

5. Requisitos Não Funcionais

5.1 Performance

- RNF01 — O dashboard deve carregar em menos de 3 segundos em conexões padrão

- RNF02 — Dados de live em tempo real devem ter latência máxima de 30 segundos
- RNF03 — O sistema deve suportar ao menos 10.000 usuários simultâneos na fase de escala

5.2 Segurança

- RNF04 — Tokens OAuth devem ser armazenados criptografados (AES-256 ou equivalente)
- RNF05 — Comunicação entre cliente e servidor deve usar HTTPS/TLS obrigatoriamente
- RNF06 — O sistema nunca deve armazenar senhas das plataformas de terceiros
- RNF07 — Logs de acesso devem ser mantidos por 90 dias para auditoria

5.3 Disponibilidade

- RNF08 — SLA mínimo de 99,5% de uptime para o plano pago
- RNF09 — O sistema deve degradar graciosamente: se uma API de plataforma falhar, os demais dados continuam sendo exibidos

5.4 Manutenibilidade

- RNF10 — Cada integração de plataforma deve ser um módulo isolado para facilitar atualizações independentes
- RNF11 — Mudanças nas APIs das plataformas devem poder ser adaptadas sem afetar o restante do sistema

6. Arquitetura Técnica Sugerida

6.1 Stack Recomendado

Camada	Tecnologia Sugerida	Justificativa
Frontend	React + TypeScript	Ecossistema maduro, componentes reutilizáveis
Backend / API	Node.js + NestJS	Alta performance para I/O, TypeScript nativo
Banco de Dados	PostgreSQL + Redis	Relacional para histórico, cache para tempo real
Autenticação	OAuth 2.0 por plataforma	Único método seguro e aceito pelas APIs
Infra / Cloud	AWS ou GCP	Escalabilidade e serviços gerenciados
Real-time	WebSocket (Socket.io)	Dados de live sem polling excessivo

6.2 Fluxo de Dados

- O usuário autentica via OAuth na plataforma de origem
- O backend recebe e armazena o token de acesso de forma segura
- Jobs periódicos consultam as APIs das plataformas e persistem os dados no banco
- O frontend consome uma API interna unificada, nunca as APIs externas diretamente
- Dados em tempo real (live ativa) são servidos via WebSocket

7. Roadmap de Desenvolvimento

Fase	Prazo Estimado	Entregas	Status
Fase 0	1 mês	Validação de APIs, setup de ambiente, autenticação OAuth	Planejado
Fase 1 — MVP	3 meses	Dashboard básico: YouTube + Twitch, métricas de seguidores e views	Planejado
Fase 2	2 meses	Integração Kick, analytics de live em tempo real, relatórios semanais	Planejado
Fase 3	2 meses	Insights inteligentes, sugestão de horários, alertas automáticos	Planejado
Fase 4	Contínuo	TikTok (na medida do possível), app mobile, plano de monetização	Futuro

8. Riscos e Mitigações

Risco	Impacto	Mitigação
APIs das plataformas mudarem sem aviso	Alto	Módulos isolados por plataforma, monitoramento de changelog das APIs
TikTok bloquear acesso aos dados de live	Médio	Tratar TikTok como feature opcional, não como core do produto
Baixa adesão de streamers pequenos	Alto	Plano gratuito robusto para aquisição orgânica via comunidade
Concorrente grande copiar a ideia	Médio	Velocidade de execução e foco em nicho (streamers ao vivo)
Custo de infraestrutura em escala	Médio	Arquitetura serverless para partes não críticas, cache agressivo

9. Modelo de Negócio Sugerido

9.1 Planos

Plano	Preço	Funcionalidades	Público-Alvo
Free	R\$ 0/mês	2 plataformas, dados básicos, histórico 30 dias	Streamers iniciantes
Pro	R\$ 49/mês	Todas as plataformas, insights, histórico 1 ano, alertas	Streamers em crescimento
Agency	R\$ 199/mês	Múltiplos criadores, relatórios white-label, API própria	Managers e agências

10. Próximos Passos Recomendados

Para transformar esta ideia em produto, recomenda-se seguir a ordem abaixo:

1. Validar as APIs do YouTube e Twitch individualmente, entendendo os limites de requisições e dados disponíveis.
2. Construir um protótipo navegável (Figma ou similar) para validar o conceito com 5 a 10 streamers reais antes de escrever código.
3. Montar um MVP funcional apenas com YouTube + Twitch e testar com beta users.
4. Registrar a empresa e proteger a marca antes de qualquer lançamento público.
5. Definir estratégia de go-to-market: comunidades de streamers, Discord, parcerias com criadores médios.

Este documento é um ponto de partida. Os requisitos devem evoluir com base em feedback real de usuários.